Learning File 2: The Logic and Algorithms

This file explains the specific algorithms used. If you understand this, you understand the "Magic."

1. The Ingestion Logic: How do we "read"?

A. Text Extraction (PyPDFLoader) A PDF is just a visual file. We use a library to scrape the raw text strings from it.

B. Recursive Chunking (The Most Important Part) We cannot save the text as one blob. We must split it.

- Algorithm: RecursiveCharacterTextSplitter
- Settings: chunk_size=1000, chunk_overlap=200.
- Logic:
    - It tries to split by paragraphs first.
    - If a paragraph is too big, it splits by sentences.
    - If a sentence is too big, it splits by words.
    - The "Overlap" (200 chars) means the end of Chunk 1 is repeated at the start of Chunk 2. This ensures we don't cut a sentence like "NOT liable" into "NOT" (Chunk 1) and "liable" (Chunk 2), which would reverse the meaning.

C. Vector Embeddings (The Math) How does the computer know that "Termination" is similar to "End of Agreement"?

- We use a "Transformer Model" (sentence-transformers/all-MiniLM-L6-v2).
- It converts text into a "Vector" (a list of 384 numbers).
- Example (Simplified):
    - "Cat" -> [0.1, 0.5, 0.9]
    - "Dog" -> [0.1, 0.6, 0.8]
    - "Car" -> [0.9, 0.1, 0.0]
- Notice "Cat" and "Dog" have similar numbers. "Car" is totally different.
- We save these lists of numbers into ChromaDB.

2. The Retrieval Logic: Cosine Similarity When you ask "Is there a termination clause?":

3. We convert your question into numbers: [0.2, 0.5, 0.8].

4. We look at all the saved chunks in ChromaDB.

5. We calculate the "Cosine Similarity" (the angle between the vectors).

6. We pick the top 3 chunks with the smallest angle (highest similarity).

7. The Generation Logic: Prompt Engineering Now we have the 3 chunks (the "Context"). We need the AI to judge them. We don't just say "Check this." We use a "System Prompt" to force behavior.

The Prompt Structure:

- Role: "You are a ruthless Senior Legal Compliance Officer." (Sets the persona).
- Task: "Analyze the provided context against the checklist item."
- Constraint: "Output purely in JSON format." (Crucial for the code to read the answer).

- Input: We inject the and the .

4. The Loop We don't send the whole checklist at once. We loop through it item by item.

- Item 1 -> Retrieve -> Analyze -> Result 1
- Item 2 -> Retrieve -> Analyze -> Result 2
- ... This ensures the AI focuses 100% of its attention on one specific risk at a time.